Routing Strategies for Ring Topologies in OMNeT++

Laboratorio 4 — Capa de Red

Álvarez, Lis Ailín; Pinilla, Lautaro; Sabino, Lucas
Grupo 23 • FaMAF/UNC

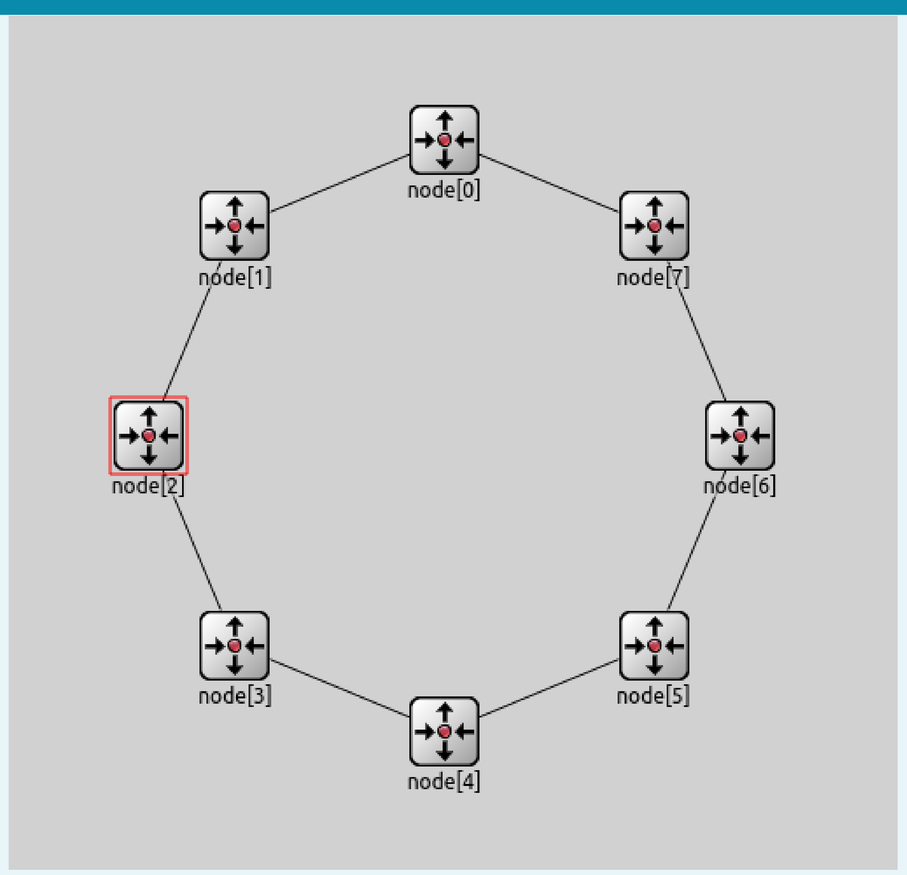
Motivación y objetivos

- **Problema:** Congestión severa por rutas asimétricas en la topología de anillo.
- **Baseline:** Reenvío estático y ciego por `toLnk[0]` (sentido antihorario).
- **Objetivo:** Diseñar un algoritmo autónomo basado en distancia mínima.
- **Herramienta:** Simulación de tráfico en OMNeT++ (demora, saltos, buffers).

Eficiencia del algoritmo propuesto

Nuestro algoritmo basado en la distancia mínima elimina la congestión estructural del anillo, balanceando la carga y reduciendo la demora media hasta un 83.5%.

Modelo de simulación



Topología en anillo — 8 nodos

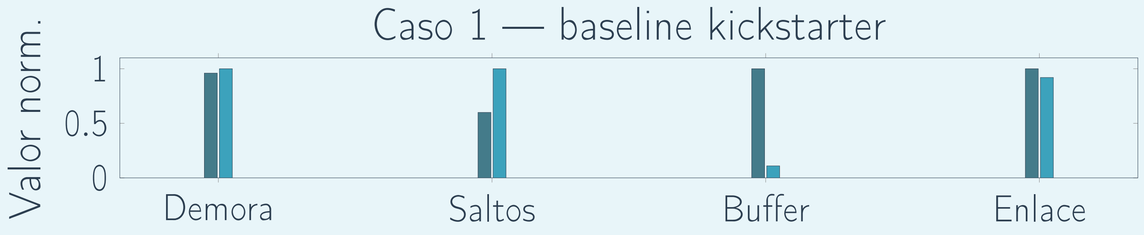
Parámetros Globales (`omnetpp.ini`):

- **Tiempo de simulación:** 200 s (`seed-set = 1`)
- **Tamaño de paquete:** 125.000 bytes (1 Mbit)
- **Generación de tráfico:** Exponencial (`interArrivalTime`)

Arquitectura interna: `app` → `net` → `lnk[0,1]`
Métricas: Demora E2E, saltos, ocupación de buffer y utilización de enlace.

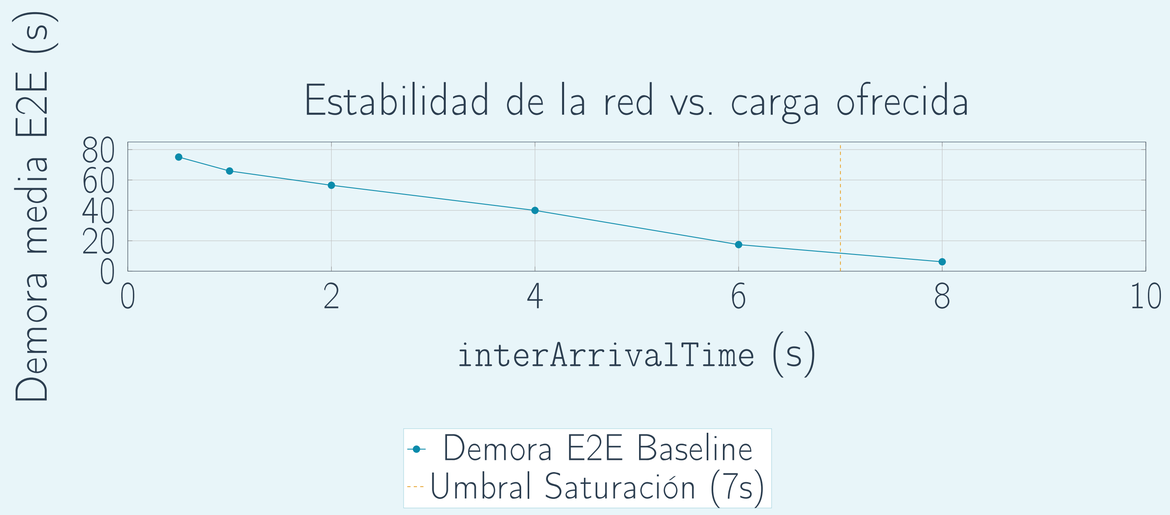
Análisis — Caso 1

Fuentes: `node[0]` y `node[2]` → `node[5]`



Métrica Global	Prom.	Máx.
Demora E2E (s)	53.25	–
Saltos	3.8	5
Ocupación Buffer (pkts)	21.8	199
Utilización Enlace (%)	29.5	98.0

Análisis — Caso 2



Cuello de Botella: *El nodo 6 acumula los 7 flujos de la red. Físicamente inestable para un IAT < 7 segundos.*

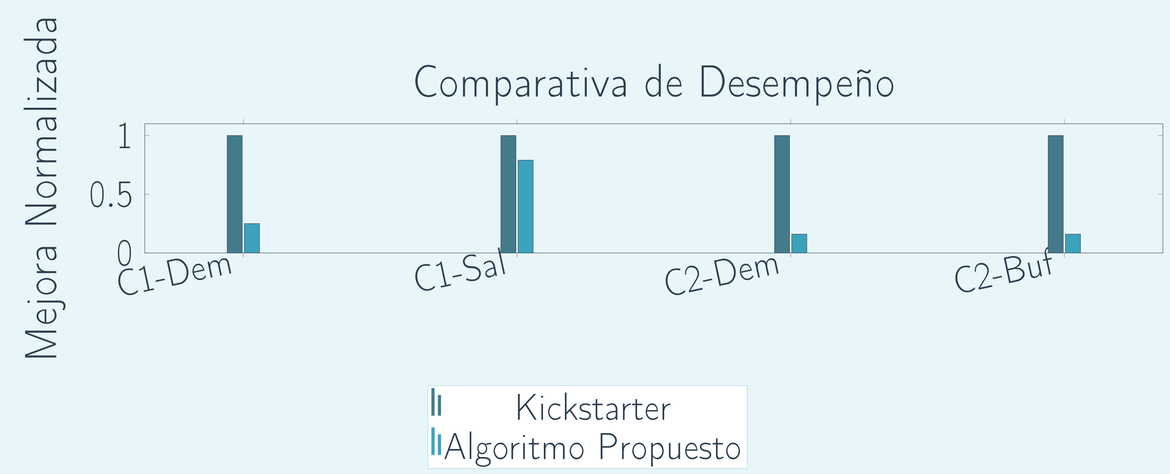
Algoritmo propuesto

- **Descubrimiento:** Utiliza paquetes de control `hello/echo` para conocer dinámicamente el tamaño de la topología (N).
- **Cálculo:** Evalúa la distancia matemática en ambos sentidos del anillo.
- **Balanceo:** Decide sobre la marcha la interfaz óptima por paquete.

```
horario = (destino - mi_id + N) % N
antihorario = (mi_id - destino + N) % N
si horario ≤ antihorario:
  enviar por lnk[1] (sentido horario)
sino:
  enviar por lnk[0] (sentido antihorario)
```

Ventaja: Autónomo. No requiere configurar constantes a mano ni utilizar iteradores trampa como `cTopology`.

Evaluación comparativa



Mejoras Caso 1:

- Demora E2E: −74.4%
- Buffer Máximo: −86.4%

Mejoras Caso 2 (IAT=4s):

- Demora E2E: −83.5%
- Buffer Máximo: −84.0%

- **Bucles de enrutamiento:** Nulos. El paquete viaja de forma monótona acercándose al destino, limitando el máximo a 4 saltos.
- **Umbral teórico de estabilidad:** mejorado de 7s a 4s al reducir la concentración máxima de 7 a 4 flujos.

Conclusiones

- El algoritmo kickstarter concentra el tráfico asimétricamente, generando un severo *queueing delay* al forzar 7 flujos a transitar por un mismo nodo, colapsando con cargas medias.
- Nuestra estrategia distribuye los flujos en ambos sentidos del anillo. Esto reduce dramáticamente la demora y aleja el umbral de estabilidad teórico de 7s a 4s, permitiendo a la red procesar de forma completamente estable intervalos de llegada (IAT) de 6s que antes colapsaban la red.
- **Trabajo futuro:** Implementar mensajes *keepalive* periódicos en reemplazo del *hello* inicial para detectar la caída de nodos y dotar a la red de tolerancia a fallos.